. First-Level Cache (Persistence Context)

The First-Level Cache is the **Session-level** cache. It is mandatory and tied to the lifecycle of a specific Hibernate Session (usually the duration of a single database transaction).

How it Works:

- **Identity Map:** Within a single transaction, Hibernate ensures that for a specific database row (ID), there is only **one** instance of that object in memory.
- **The Benefit:** If you call repository.findById(101) multiple times in the same method, Hibernate only executes a SELECT statement the **first time**. For subsequent calls, it simply returns the object already sitting in the cache.

Example Scenario:

Imagine you are processing a log batch where multiple logs belong to the same "Service."

1. You fetch Service A for Log 1. (SQL: SELECT ...)
2. For Log 2, you fetch Service A again.
3. **Result:** Hibernate returns the cached Service A from the First-Level Cache. This prevents 99 redundant SQL calls if a batch has 100 logs for the same service.

---

2. Dirty Checking

Dirty checking is the mechanism by which Hibernate automatically detects if an entity has been modified and needs to be synchronized with the database.

How it Works:

1. **Snapshoting:** When an object is loaded into the Persistence Context, Hibernate keeps a "snapshot" of its initial state.
2. **Comparison:** When the transaction is about to commit (the **Flush** phase), Hibernate compares the current state of the object with that original snapshot.
3. **Auto-Update:** If they are different (the object is "dirty"), Hibernate automatically generates and executes an UPDATE SQL statement.

The Code Benefit:

You don't need to call .update() or .save() explicitly for objects that are already "Managed."

Java

```
@Transactional
public void updateStatus(Long id) {
   User user = repository.findById(id).get(); // Now Managed
   user.setStatus("ACTIVE");
   // No need to call repository.save(user)!
   // Hibernate's dirty checking will trigger the UPDATE at the end of the method.
}
```

---

3. Transaction Management

Transaction management in Spring/Hibernate ensures the **ACID** properties of your business logic.

The Proxy Mechanism:

When you use @Transactional, Spring creates a **Proxy** around your class.

- **Start:** Before your method runs, the proxy starts a transaction and opens a Hibernate Session.
- **Success:** If the method completes without an exception, the proxy tells Hibernate to **Flush** (execute pending SQL) and **Commit** the transaction.
- **Failure:** If a RuntimeException occurs, the proxy calls **Rollback**, and all changes (even those already sent to the DB) are undone.

The "Self-Invocation" Trap:

A common interview question: "Why does @Transactional fail if I call the method from within the same class?"

- **Answer:** Because you are bypassing the **Proxy**. The call goes directly to the internal method, and the "Transaction Start" logic is never triggered.

Transaction Propagation (Briefly):

- **REQUIRED (Default):** "I need a transaction. If one exists, I'll join it; otherwise, I'll start a new one."

- **REQUIRES_NEW:** "I don't care about the current transaction. Pause it, and start a completely fresh one for me." (Useful for logging errors to a DB without rolling back the log when the main task fails).

Summary for Your Interview

Feature	Technical "Why"
First-Level Cache	Prevents redundant SELECTs within the same transaction.
Dirty Checking	Simplifies code; ensures only modified fields/objects are updated.
Flush	The bridge between memory and DB. Happens at the end of the transaction.
Rollback	Ensures data integrity; defaults to RuntimeException only.